

MWORKS 2025a培训（二）

解锁Syslab代码调试新技能

孔德才

苏州同元软控信息技术有限公司

2025年3月7日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

传播或以其他方式使用

MWORKS.Syslab 2025a 更新概览

01 更好用的Syslab

- 软件使用全旅程体验提升

02 更快的Syslab

- 持续改进“启动慢、首次运行慢”问题
- 释放Julia高性能潜力，持续优化函数算法实现

03 更全面的Syslab

- 数学、统计与优化功能升级
- 图像处理工具箱全新发布
- 代码生成功能升级

详见：

B站 发布会直播回放：<https://www.bilibili.com/video/BV1aUraYREjj>

公众号 发布会直播回顾：<https://mp.weixin.qq.com/s/6JCJTnuGQbYqMv4N3Ylj3Q>

目录

1. 实时脚本

2. 函数构建及调用

3. 代码调试

1. 实时脚本

题目：苏州某地安装有太阳能电池板，拟对比**实际产量**与**理论产量**，验证其性能



已知数据：

- ◆ 当地**太阳赤纬角**约为23° 24'，**纬度**为 31° 30'
- ◆ **电池板面积**为 270 m²，最大输出为 207 kw
- ◆ 电池板**某月份**的**产量数据**，提供Excel文件

理论公式：

- ◆ 太阳与电池板之间的夹角公式

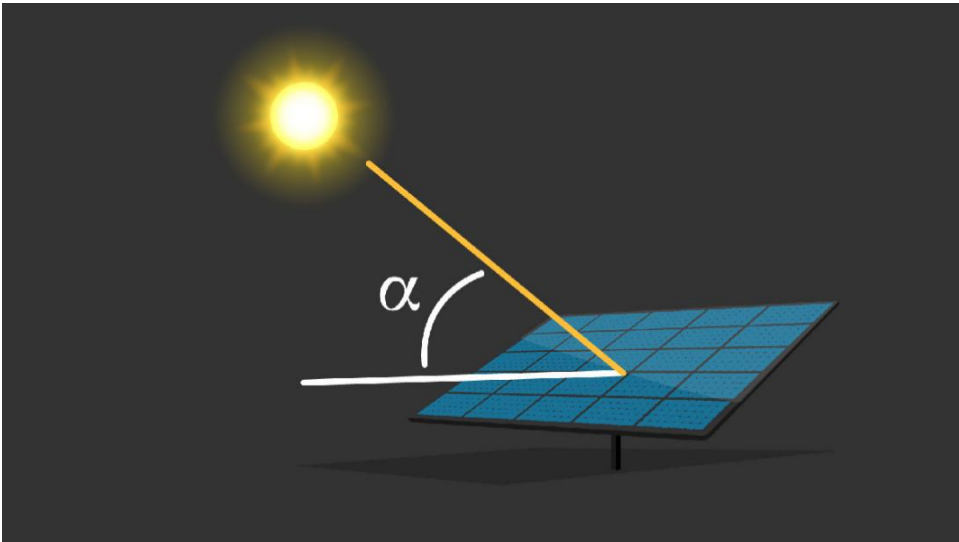
$$\sin(\alpha) = \sin(\underbrace{\delta}_{\text{太阳赤纬角}}) \cdot \underbrace{\sin(\underbrace{\varphi}_{\text{纬度}})}_{\text{太阳赤纬角}} + \cos(\underbrace{\delta}_{\text{太阳赤纬角}}) \cdot \cos(\underbrace{\varphi}_{\text{纬度}}) \cdot \cos(15 \cdot (\underbrace{t}_{\text{时间}} - 12))$$

- ◆ 考虑大气层影响的经验系数公式

$$T_{inc} = 1.4883 \times 0.7^{\sin(\alpha) - 0.678}$$

- ◆ 电池板产量公式

$$P = \underbrace{S}_{\text{电池板面积}} \cdot \sin(\alpha) \cdot T_{inc}$$

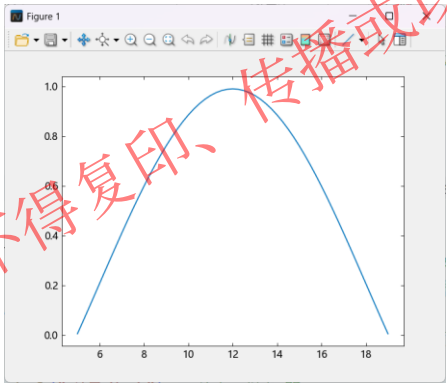


1. 实时脚本

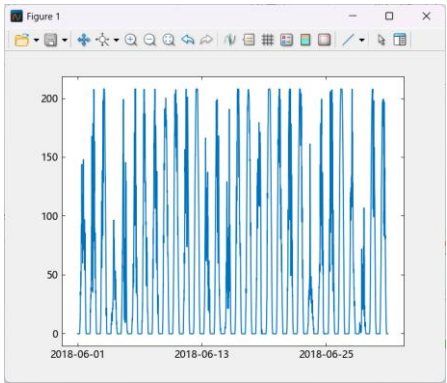
通过构建.jl脚本文件方式完成任务

```
1 ###
2 dec = 23.4 # 太阳偏角: 23° 24'
3 lat = 31 + 30 / 60 # 纬度: 31° 30'
4 dec = deg2rad(dec) # 将角度转换为弧度
5 lat = deg2rad(lat)
6 t = 5:0.25:19 # 默认时间从早上5点开始, 到19点结束
7 # 计算太阳与电池板之间的夹角的函数
8 sunangle = sin(dec) * sin(lat) .+ cos(dec) .* cos(lat) .* cosd.(15 .* (t.-12))
9 plot(t, sunangle) # 绘制夹角与时间的曲线
10
11 T_inc = 1.4883 * 0.7 .^ (sunangle .^ -0.678) # 考虑大气层影响
12 production_theory = 270 * T_inc .* sunangle # 计算电池板理论产量
13 k = find(production_theory .> 207) # 考虑电池板最大输出为207kw
14 production_theory[k] .= 207
15 plot(t, production_theory) # 绘制理论产量与时间的曲线
16 xlabel("时间") # 添加X轴标签
17 ylabel("产量(kw)") # 添加Y轴标签
18 title("理论数据") # 添加图标题
19
20 ##
21 production = readtable("太阳能电池板产量数据.xlsx") # 读取实际监测数据
```

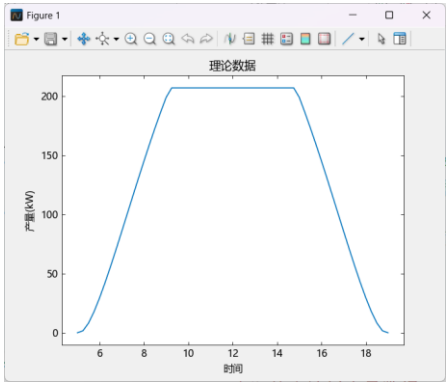
是否有另一种形式也可以完成呢?



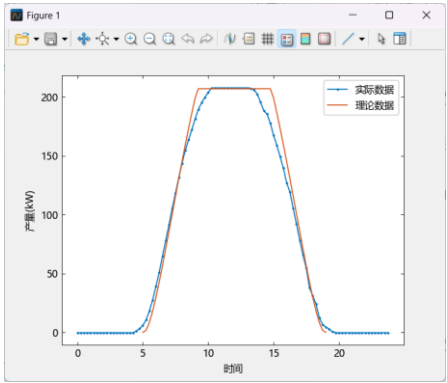
太阳与电池板之间的夹角与时间曲线



电池板整月实际产量与时间曲线



电池板理论产量与时间曲线

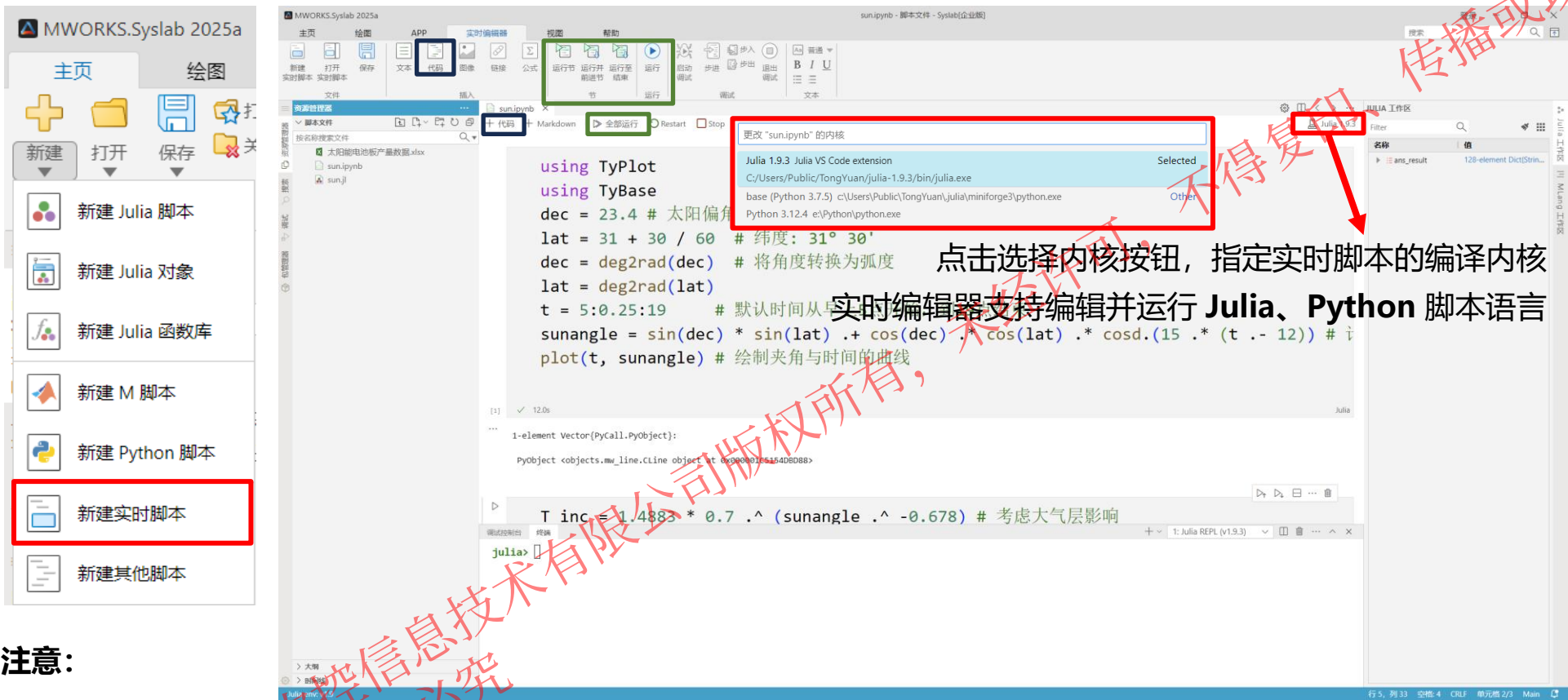


电池板实际产量和理论产量与时间曲线



1. 实时脚本

支持用户在统一的文档环境中将代码与嵌入式输出、格式化文本、方程和图像组合到一起，生成可交互式记事本，并与他人分享



新建Julia / Python代码:

- 方式1:  代码
- 方式2: + 代码

代码运行:

- **运行节:** 运行当前节
- **运行并前进节:** 运行当前节并前进到下一节
- **运行至结束:** 从当前节运行至代码结束
- **运行或全部运行:** 运行全部代码
- **停止:** 终止运行进程

注意:

新建或打开实时脚本，启动实时编辑器功能

Tips: 此处编写的Julia脚本默认不加载任何工具箱，需要自己using XXX函数库，包括同元函数库

@苏州同元软控信息技术有限公司
版权所有，侵权必究

目录

1. 实时脚本

2. 函数构建及调用

3. 代码调试

2.1 函数构建及调用

Syslab 2025a 提升代码编辑易用性

- 函数库加载提示：对于从资源管理器中打开的Julia脚本，会检查该文件中未定义的库函数，以波浪线提示，并提供快速修复功能

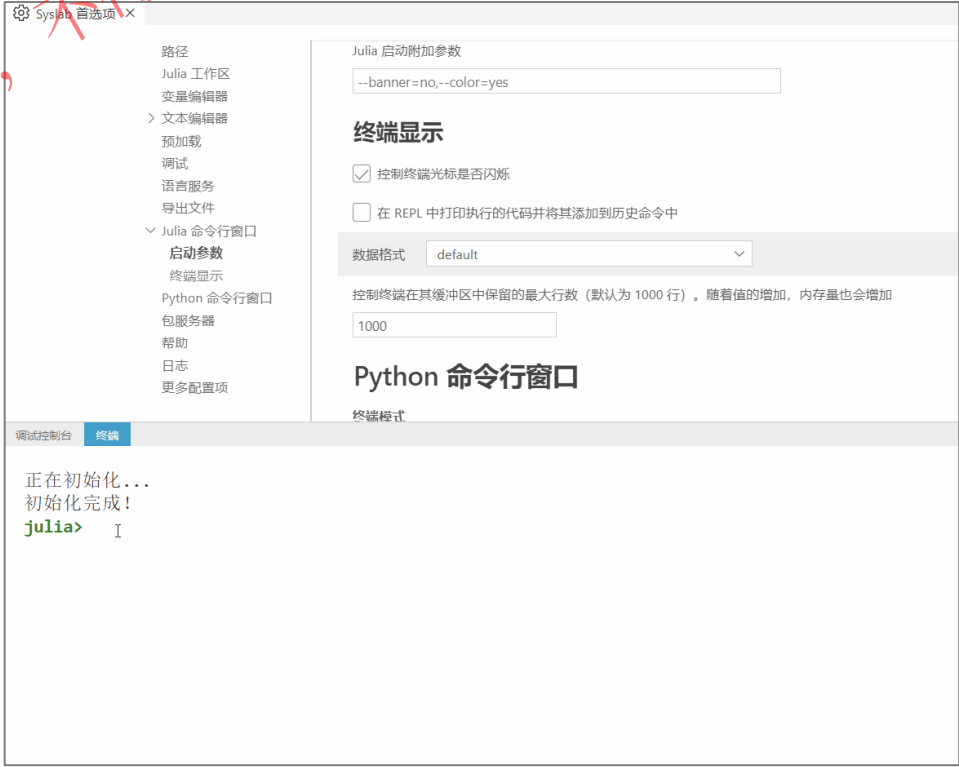
函数库加载提示.jl ×

```
1
2 A = magic(4)
3 x = 0:pi/100:2*pi;
4 y = sin.(x);
5 plot(x, y)
6 xlabel("x")
7 ylabel("sin(x)")
8 title("Plot of the Sine Function")
9
10
```

调试控制台 终端

正在初始化...
初始化完成!
julia>

- 终端数据格式显示设置：首选项里提供3种设置，分别为自适应(default)、长固定(long)、短固定(short)，可根据需求进行选择



2.1 函数构建及调用

示例：计算A、B、C多个矩阵的乘积

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad C = \begin{bmatrix} 9 \\ 10 \end{bmatrix}$$

计算多个矩阵的乘积

```
function multiply_matrices_test(matrices::Vector{Array{Float64}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

定义矩阵A、B、C

A = [1.0 2.0; 3.0 4.0]

B = [5.0 6.0; 7.0 8.0]

C = [9.0; 10.0]

matrices = [A, B, C]

result = multiply_matrices_test(matrices)

主函数与子函数在同一脚本

输出结果为：

2-element Vector{Float64}:
391.0
887.0

2.1 函数构建及调用

示例：计算A、B、C多个矩阵的乘积

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \quad C = \begin{bmatrix} 9 \\ 10 \end{bmatrix}$$

```
# 计算多个矩阵的乘积
function multiply_matrices_test(matrices::Vector{Array{Float64}})
    result = matrices[1]
    for i in 2:length(matrices)
        result = result * matrices[i]
    end
    return result
end
```

```
include("multiply_matrices.jl")
A = [1.0 2.0; 3.0 4.0]
B = [5.0 6.0; 7.0 8.0]
C = [9.0; 10.0]
matrices = [A, B, C]
result = multiply_matrices_test(matrices)
```

主函数与子函数在不同脚本



形成子函数文件



构建主函数文件

2.1 函数构建及调用

include("函数文件路径") 实现函数脚本调用

□ 资源管理器打开至函数文件所在目录



• 路径 ✓

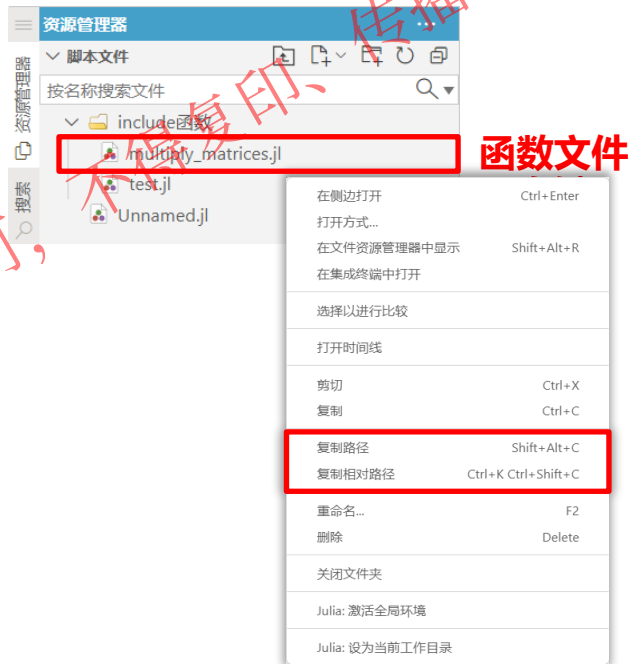
C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

• 相对路径 ✓

multiply_matrices.jl

提示: pwd() 命令也可显示当前文件夹的路径, 通过拼接构成绝对路径 include(pwd()*"/multiply_matrices.jl")

□ 资源管理器未打开至函数文件所在目录



• 路径 ✓

C:/Users/TR/Desktop/.../脚本文件/include函数/multiply_matrices.jl

• 相对路径 ✗

include函数/multiply_matrices.jl

2.1 函数构建及调用

注意事项

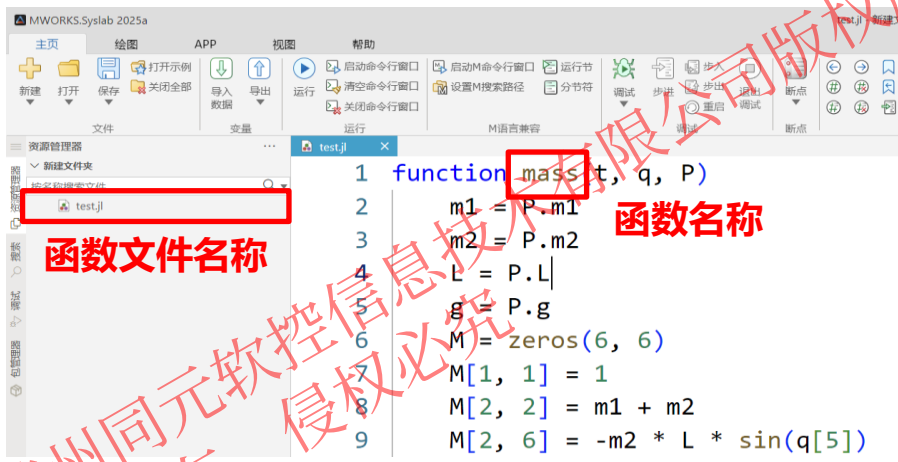
□ 关于函数文件名称

M软件:

- 函数文件名称必须与文件中定义的主函数名称完全一致
例如: 若函数定义为 `function y = CalSum(x)`, 则文件必须保存为 `CalSum.m`, 否则会报错

Syslab:

- 函数文件名称与文件中定义的函数名称无关联



Syslab

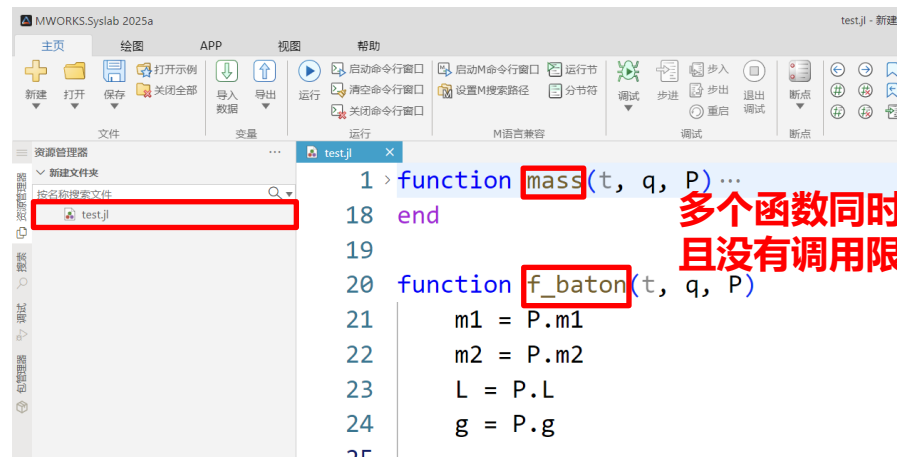
□ 关于函数文件中定义多个函数

M软件:

- 函数文件支持一个主函数+多个子函数的结构
- 每个函数文件只能有一个主函数, 其名称必须与文件名一致
- 在主函数之后, 可以定义多个子函数, 子函数仅在该文件内可见, 无法被外部直接调用

Syslab:

- 函数文件可定义多个函数, 没有主函数或子函数的严格限制



多个函数同时存在
且没有调用限制

Syslab

2.2 工具箱安装和卸载

在Syslab中可以通过包管理器，管理注册库与开发库，安装和卸载工具箱

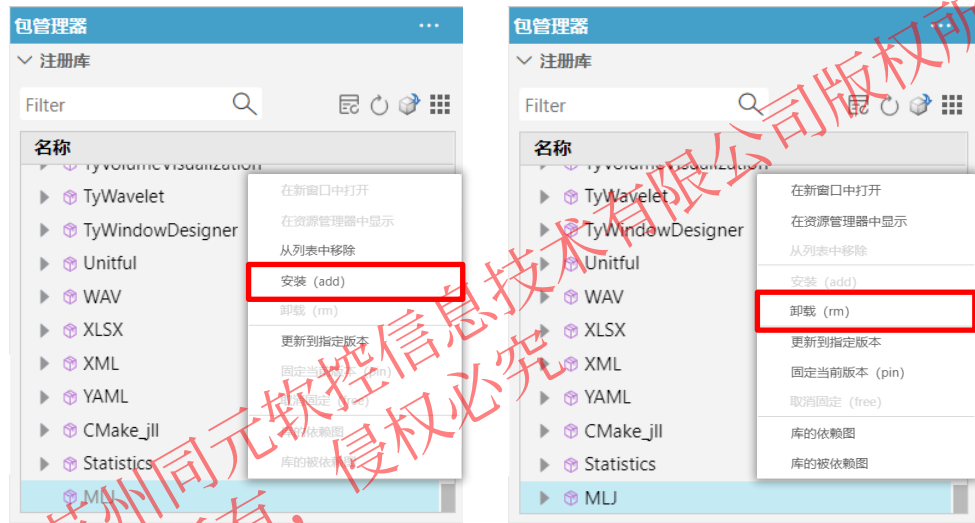
方式1：可视化操作

安装操作步骤：

1. 注册库中点击“添加包”
2. 搜索第三方工具箱，选择对应工具箱，添加至注册库列表
3. 在注册库列表中右击，选择“安装”

卸载操作步骤：

1. 注册库中右击包，选择卸载



方式2：文本操作

安装操作步骤：

1. 在REPL中输入] 进入pkg模式(退出pkg模式可使用 Backspace或Ctrl + C)
2. 在pkg模式中输入： add 第三方工具箱名

卸载操作步骤：

1. 在pkg模式中输入： remove 第三方工具箱名

```
(@v1.9) pkg> add MLJ
```

安装

```
(@v1.9) pkg> remove MLJ
```

卸载

说明：

- 同元商业工具箱无需安装，可直接使用

目录

1. 实时脚本

2. 函数构建及调用

3. 代码调试

3. 代码调试

代码编写很难一次成功，往往需要进行代码调试，列举2个常见的错误情况

□ 情况1-代码运行报错

```
test.jl × multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0] 逗号改为分号
6 matrices = [A; B; C]
7 result = multiply_matrices_test(matrices)
```

运行报错

```
调试控制台 终端
julia> 正在运行 test.jl
ERROR: ArgumentError: number of columns of each array must match (got (2, 2, 1))
Stacktrace:
 [1] _typed_vcat{#unused#::Type{Float64}, A::Tuple{Matrix{Float64}, Matrix{Float64}, Vector{Float64}}}
    @ Base .\abstractarray.jl:1689
 [2] typed_vcat
    @ .\abstractarray.jl:1703 [inlined]
 [3] vcat{::Matrix{Float64}, ::Matrix{Float64}, ::Vector{Float64}}
    @ Base .\array.jl:1958
 [4] top-level scope
    @ c:\Users\TR\Desktop\include函数\test.jl:6
```

报错原因：列数不匹配

出错代码位置：第6行

□ 情况2-代码运行不报错，结果错误

```
test.jl × multiply_matrices.jl ×
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::Vector{Matrix{Float64}})
3     result = matrices[1]
4     for i in 1:length(matrices)
5         result = result * matrices[i]
6     end
7     return result
8 end
```

结果错误

```
调试控制台 终端
```

```
julia> 正在运行 test.jl
2-element Vector{Float64}:
 2165.0
 4721.0
```

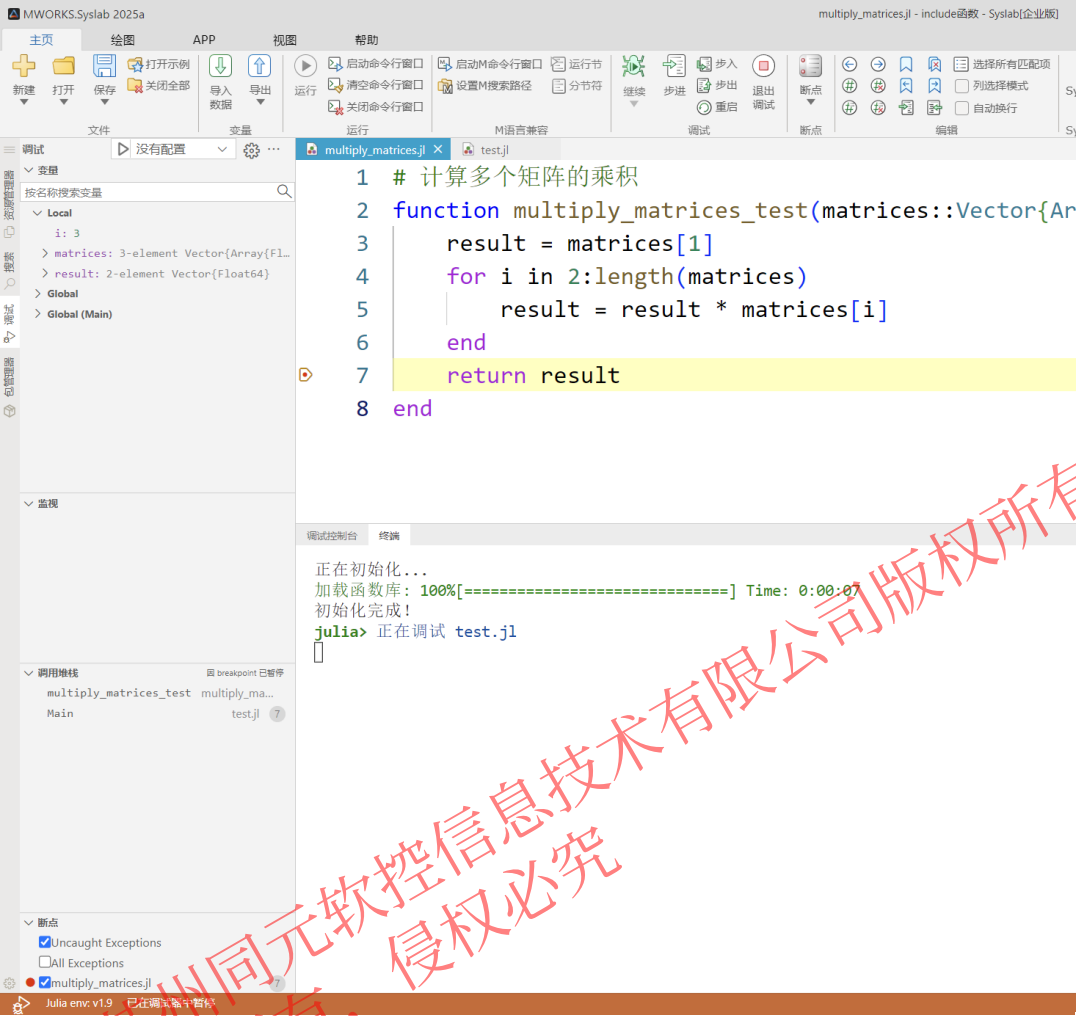
正确结果为：
2-element Vector{Float64}:
391.0
887.0

说明：在此例中，情况1无需调试，情况2需调试



3. 代码调试

Syslab提供代码调试器，支持对代码文件的**单步调试**、**断点调试**、**添加监视**、**查看调用堆栈**等，并提供**交互式调试控制台**



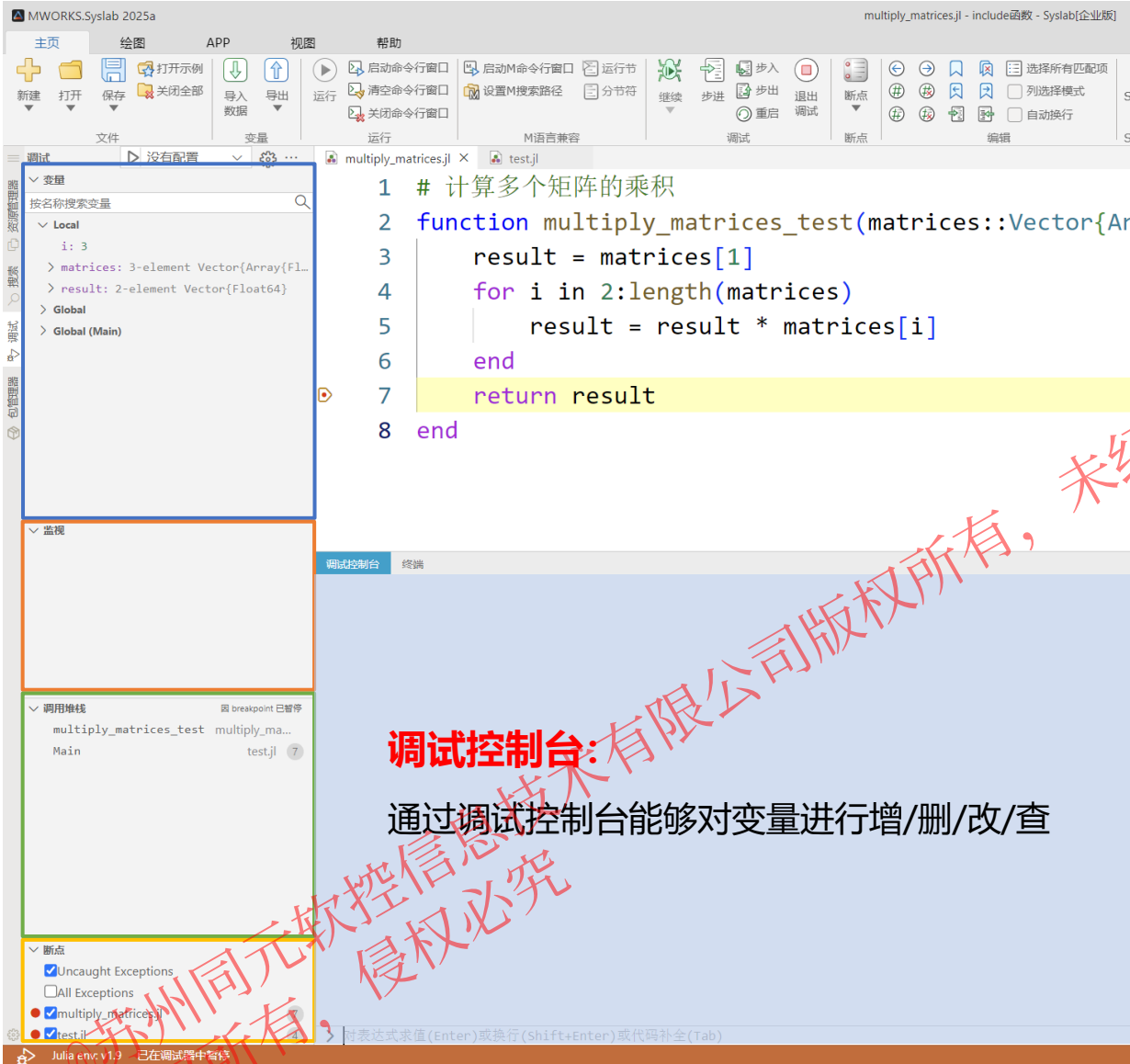
操作步骤:

1. 在脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

说明:

- **继续(F5)**: 继续运行调试
- **步进(F10)**: 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止
- **步入(F11)**: 单步执行遇到子函数就进入并且继续单步执行
- **步出(Shift+F11)**: 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数
- **重启(Ctrl+Shift+F5)**: 重新启动调试
- **退出调试(Shift+F5)**: 停止调试

3. 代码调试



- 说明:**
- **变量:**
 - Local: 函数局部变量的值
 - Global: 函数全局变量的值, 变量前具有global前缀
 - Global(Main): 主程序全局变量的值
 - **监视:** 输入变量名, 监视变量的变化
 - **调用堆栈:** 用于了解函数间的调用关系和逻辑
 - **断点:** 模型中所有断点的信息

关于**变量作用域**相关内容, 可查看帮助文档



3. 代码调试

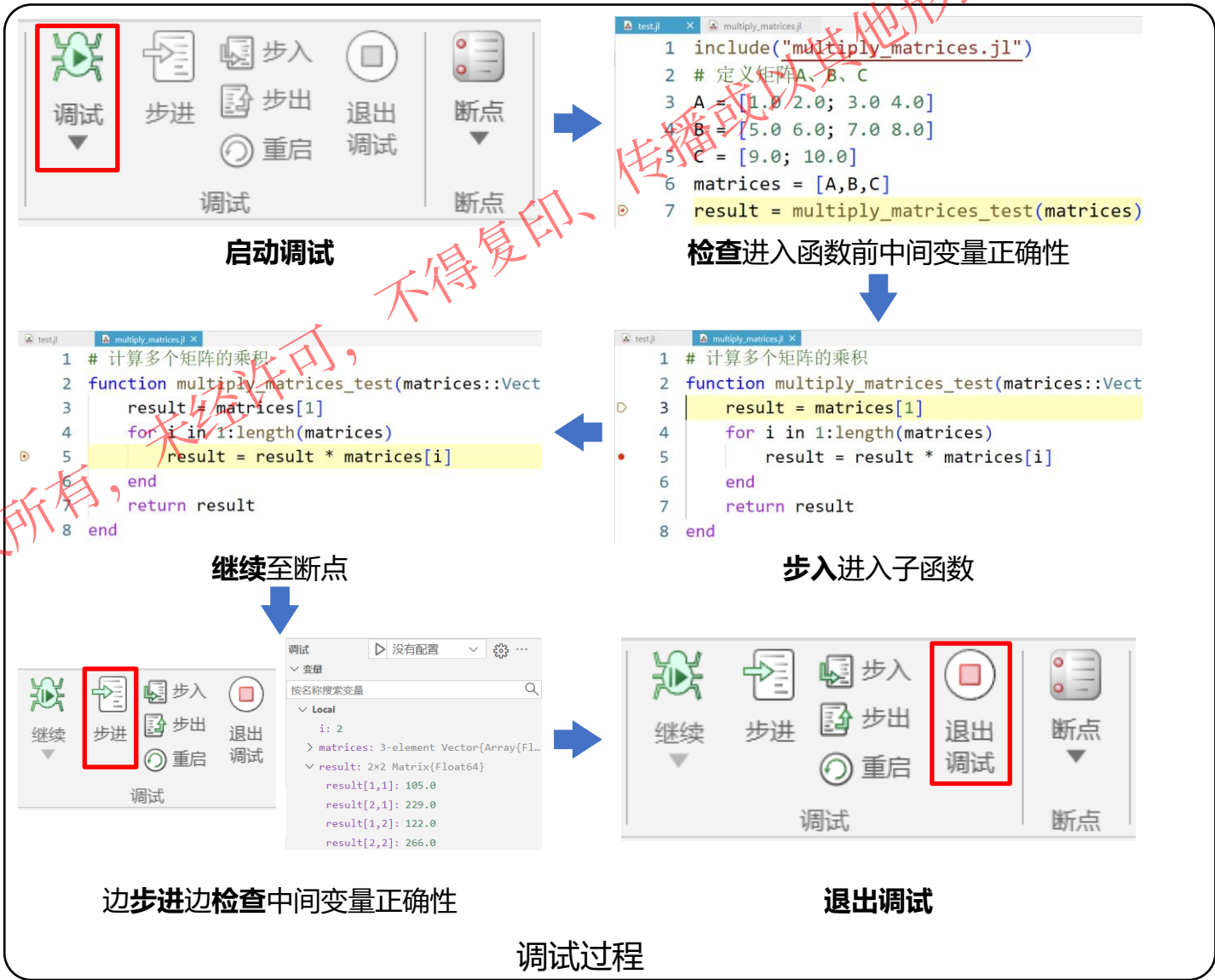
操作步骤:

- 1. 在脚本打上断点
- 2. 点击菜单栏中"启动调试"或F5进入DeBug模式

```
test.jl x multiply_matrices.jl
1 include("multiply_matrices.jl")
2 # 定义矩阵A、B、C
3 A = [1.0 2.0; 3.0 4.0]
4 B = [5.0 6.0; 7.0 8.0]
5 C = [9.0; 10.0]
断点1 matrices = [A,B,C]
• 7 result = multiply_matrices_test(matrices)
```

```
test.jl x multiply_matrices.jl x
1 # 计算多个矩阵的乘积
2 function multiply_matrices_test(matrices::
3     result = matrices[1]
断点2 for i in 1:length(matrices)
5         result = result * matrices[i]
6     end
7     return result
8 end
```

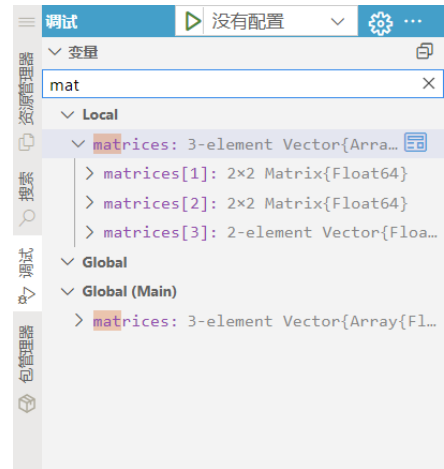
打断点



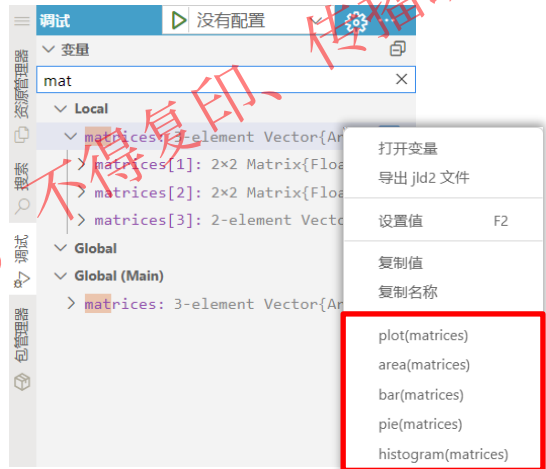
3. 代码调试

Syslab 2025a 提升代码调试易用性

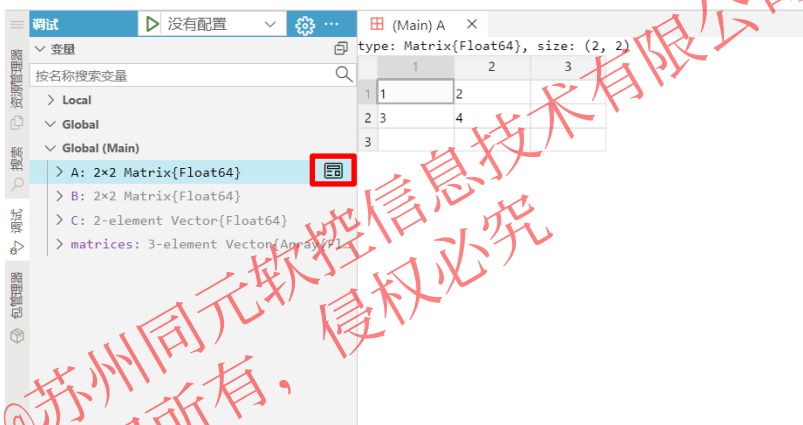
支持变量搜索



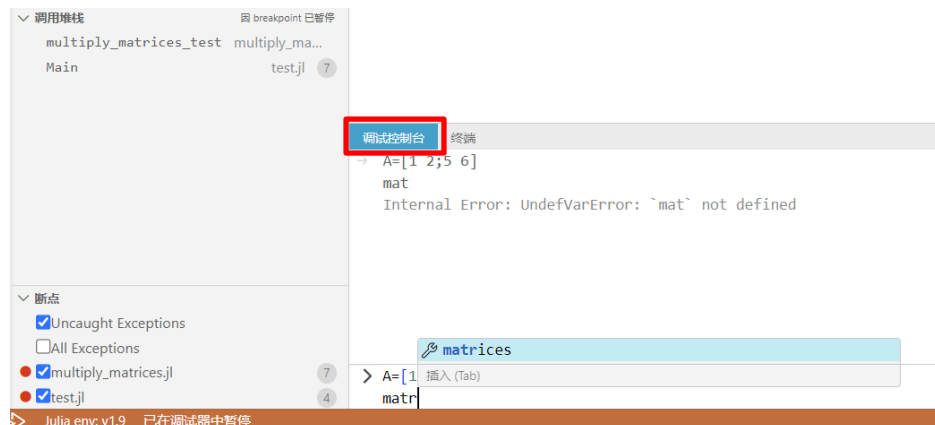
支持变量右键直接绘图



支持查看变量表格视图



调试控制台支持换行输入、代码补全



3. 代码调试

支持程序计时及程序性能采样分析

提供多种宏调用的方式，支持对表达式计时和查看内存分配情况：

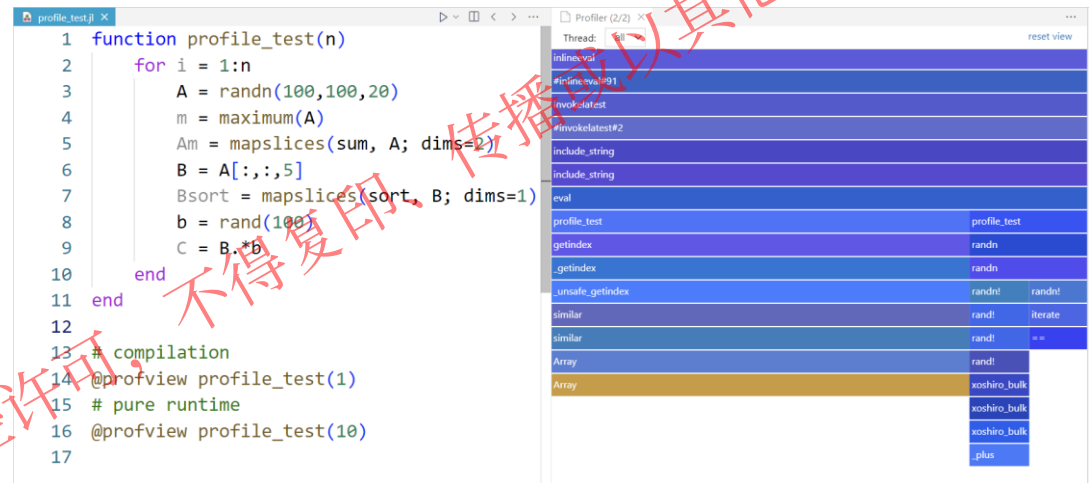
- `@time`对表达式计时和查看内存分配：运行时间；内存分配次数；总计分配的内存数；整理内存所用的时间
- `@elapsed`对表达式计时，以浮点数的形式返回执行所用的秒数。
- `@btime`提供在基准测试期间测量的最小运行时间。
- `@benchmark`用于对Julia表达式进行基准测试。其运行结果主要包括：运行次数、最小用时、最大用时、中位数用时、平均用时，以及占用内存等
- `@profview`对代码进行采样分析，并输出火焰图

```
time_test.jl
1 x = rand(10,10);
2
3 @time x * x; # 第一次存在编译时间
4
5 @time begin
6     x * x
7 end;
```

调试控制台 终端

```
julia> 正在运行 time_test.jl
0.003447 seconds (57 allocations: 3.625 KiB, 96.75% compilation time)
0.000011 seconds (1 allocation: 896 bytes)
```

@time对表达式计时和查看内存分配



性能采样分析结果

Syslab使用手册

全局搜索

目录

- 使用 Syslab
- Julia 语言
- Julia 语言概览
- Julia 高性能编程**
 - 核心思路
 - 性能分析
 - 常见性能优化
 - 数组
 - for 循环优化
 - 类型稳定
 - 数值类型
 - 多重派发
 - 内存优化
 - 内置函数使用
 - 外部语言接口

Julia 高性能编程

2025a

Julia 是一门灵活的动态语言，适合用于科学计算和数值计算，并且性能可与传统的静态类型语言媲美。写出能用的 Julia 代码很容易，但写出高性能的 Julia 代码则需要长时间的训练，以及充足的知识储备。本节主要介绍 Julia 性能优化指南，帮助用户写出高性能代码。

核心思路

- 检查性能热点
- 写对编译器友好的代码
 - 确保类型稳定
 - 确保结构体或矩阵的元素类型是具体类型
- 避免不必要的内存操作
- 避免不必要的数据计算
- 使用高效的算法和数据结构
- 广播和向量化编程并不一定是好的
- 更好地利用硬件的计算能力
- 在单线程足够高效后再考虑多线程，因为否则可能会带来负优化

性能分析

帮助文档-Julia高性能编程经验



建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听